Binary Intrusion Detection Pipeline

CIC IDS 2017 Dataset

Yemi Oluwadare

February 2026

Contents

Assumptions **3**

1. Introduction **4**

2. Problem Definition **5**

3. Dataset and Preprocessing **6**
3.1 Dataset Description . . . 6
3.2 Temporal Data Partitioning . . . 6
3.3 Column Standardisation and Identifier Removal . . . 6
3.4 Handling Missing and Non-finite Values . . . 6
3.5 Streaming and Memory-aware Processing . . . 7
3.6 Class Imbalance . . . 7

4. Experimental Design and Training Strategy **8**
4.1 Evaluation Objectives . . . 8
4.2 Baseline Model . . . 8
4.3 Model Selection Rationale . . . 8
4.4 Training Regime and Data Access Strategy . . . 8
4.5 Hyperparameter Configuration . . . 8
4.6 Validation and Test Protocol . . . 9
4.7 Performance Metrics . . . 9
4.8 Reproducibility and Artefact Preservation . . . 9

5. Implementation and System Architecture **10**

6. Results **11**
6.1 Summary Metrics . . . 11
6.2 Confusion Matrices . . . 11
6.3 ROC Curve . . . 12
6.4 Feature Importance and Interpretation . . . 13
Feature Robustness Experiment . . . 13

7. Discussion and Model Comparison **14**

8. Feature Analysis and Interpretability **15**
8.1 Linear Model Coefficients (SGD Logistic Regression) . . . 15
8.2 Random Forest Feature Importance . . . 15
8.3 Security Interpretation . . . 15

9. Limitations and Future Work **16**
9.1 Simulation Bias and Dataset Representativeness . . . 16
9.2 Temporal Shift and Concept Drift . . . 16
9.3 Adversarial Manipulation . . . 16
9.4 Scalability Constraints . . . 16
9.5 Future Work . . . 16

10. Reproducibility and Artefacts **17**
10.1 How to Reproduce the Results 17

11. Conclusion **18**

Appendix A: Figure Index **19**

Appendix B: Glossary **21**

References **23**

Assumptions

This submission implements a reproducible binary intrusion detection pipeline using flow-level features from CIC IDS 2017. The assumptions below are limited to what is required to make the pipeline reproducible on commodity hardware.

- The CIC IDS 2017 CSV files are obtained from an authoritative distribution and stored unmodified locally prior to preprocessing.
- Labels provided by the dataset are treated as ground truth for supervised learning, acknowledging that labelling in security datasets may not fully represent real enterprise ambiguity (Sommer and Paxson 2010).
- The evaluation design prioritises temporal separation (train and validation from earlier days, test from a later day) as a proxy for distribution shift.
- Commodity hardware constraints require streaming preprocessing and incremental training for at least one model.

1. Introduction

Modern computer networks operate under high traffic volumes, heterogeneous protocols, and evolving threat landscapes. Traditional intrusion detection systems (IDS) have historically relied on signature-based techniques, where known attack patterns are matched against observed traffic. While effective for previously identified threats, such systems generalise poorly to novel or obfuscated attacks and require continuous manual updating. These limitations have driven increasing adoption of data-driven and machine learning-based intrusion detection methods that model benign and malicious behaviour directly from network telemetry (Sommer and Paxson 2010).

In this work, intrusion detection is framed as a supervised binary classification task, where each network flow is labelled as either benign or indicative of a distributed denial-of-service (DDoS) attack. The objective is to detect malicious flows while controlling false positives, reflecting the operational trade-off between missed attacks and alert fatigue.

A further challenge arises from the non-stationary nature of network traffic. Changes in user behaviour, infrastructure, and attacker strategy can alter the statistical relationship between features and labels over time. As a result, models that perform well on in-window validation data may degrade when applied to traffic captured under different conditions. This work therefore emphasises generalisation across temporally separated test data as a proxy for deployment realism, rather than relying solely on random splits of a single capture window.

Two contrasting modelling approaches are evaluated: a linear classifier trained using stochastic gradient descent (SGD-based logistic regression), and a non-linear ensemble model based on random forests. This comparison highlights trade-offs between computational efficiency, interpretability, and representational capacity in large-scale, flow-based intrusion detection.

2. Problem Definition

The task is a supervised binary classification problem on labelled flow-level telemetry: benign (0) versus DDoS (1). Friday is held out as a temporally separated test set to approximate deployment shift. Although CIC IDS 2017 contains multiple attack categories, a binary formulation supports clearer analysis of false negatives and temporal generalisation.

Let $x_i \in \mathbb{R}^d$ denote the feature vector associated with the i-th network flow and $y_i \in \{0, 1\}$ the corresponding class label, where 0 represents benign traffic and 1 denotes DDoS activity. The learning objective is to estimate a function $f : \mathbb{R}^d \to \{0, 1\}$ that minimises misclassification risk while maintaining sensitivity to the minority (attack) class.

3. Dataset and Preprocessing

3.1 Dataset Description

This study uses the CIC IDS 2017 dataset, a widely adopted benchmark for evaluating machine learning approaches to network intrusion detection. The dataset consists of flow-level traffic captured in a simulated enterprise environment, where benign user activity is interleaved with scripted attack scenarios, including distributed denial-of-service (DDoS) events (Sharafaldin, Lashkari, and Ghorbani 2018). Each flow is represented by a fixed-length feature vector comprising protocol attributes, packet statistics, and temporal measurements derived from raw packet captures.

However, the controlled nature of the environment introduces potential bias, as traffic patterns and attack behaviours reflect a specific topology and workload profile rather than the full diversity of real deployment contexts.

3.2 Temporal Data Partitioning

Traffic from Monday through Thursday is used to construct the training and validation sets, while traffic from Friday is reserved as a temporally distinct test set. This design evaluates generalisation beyond the statistical structure of the training window rather than performance on random splits drawn from the same capture period.

Within the Monday-Thursday subset, a stratified split preserves class proportions between training and validation sets.

3.3 Column Standardisation and Identifier Removal

Initial inspection revealed inconsistent header formatting across CSV files, including leading whitespace in the label column. All column names were standardised to ensure deterministic feature selection and reliable label extraction.

Several raw columns are identifiers rather than behavioural measurements, including timestamps, addressing information, and flow identifiers. While predictive in closed-world benchmarks, these fields create shortcut-learning risk and are unstable across environments. All non-numeric identifiers were therefore excluded, retaining only numeric, flow-derived statistical attributes. The final feature space comprised 80 numeric features per flow.

3.4 Handling Missing and Non-finite Values

Missing and non-finite values were concentrated in rate-based features such as *Flow Bytes/s* and *Flow Packets/s*, which become undefined when flow durations approach zero. These values were replaced with numerically safe representations and handled consistently to ensure downstream models always receive finite numeric matrices. The distribution of affected values was logged to preserve traceability from raw data characteristics to training inputs.

3.5 Streaming and Memory-aware Processing

The training window contains over two million flows, making full in-memory preprocessing infeasible on commodity hardware. Preprocessing was therefore implemented using chunk-based streaming, with each partition independently cleaned and written to disk as a processed dataset.

This design mirrors operational security data pipelines, where telemetry is handled as a stream rather than a static table, and separates deterministic preprocessing from model fitting to enable repeatable experimentation.

3.6 Class Imbalance

The dataset exhibits pronounced class imbalance, with approximately 87% benign flows and 13% DDoS flows in the training set. Under these conditions, a majority-class classifier can achieve high accuracy while failing to detect attacks (He and Garcia 2009). Evaluation therefore prioritises macro-averaged F1, precision, and recall rather than accuracy alone.

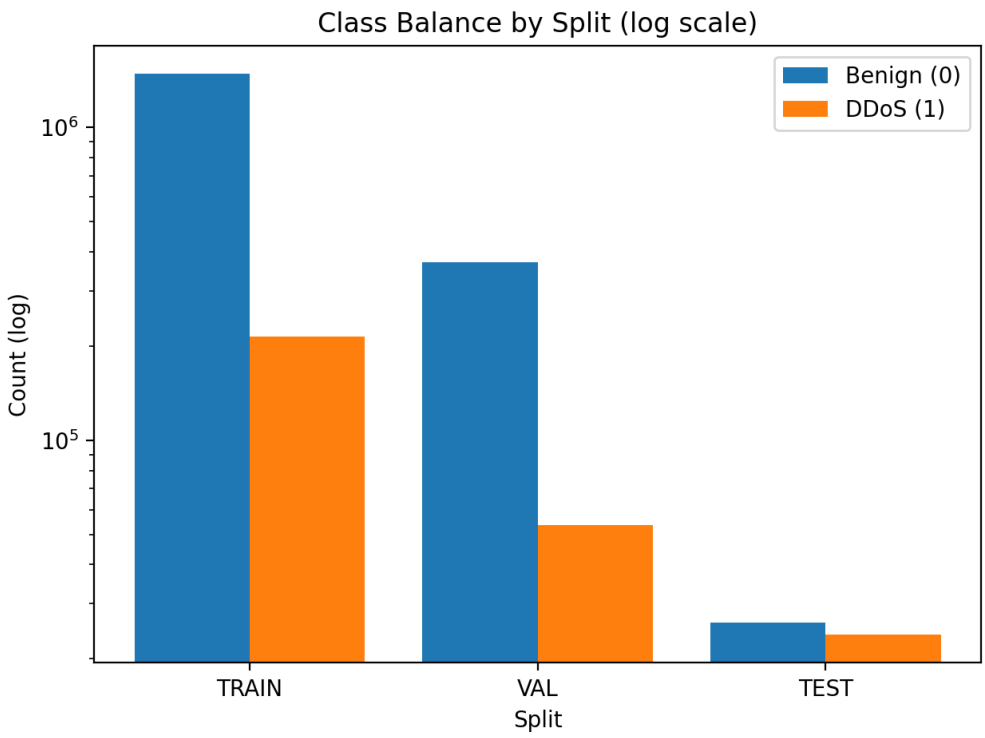

Figure 1: Class balance by split (valid labels). Friday shows a marked shift relative to Monday-Thursday.

Figure 1 confirms pronounced imbalance in the Monday-Thursday window (approximately 87% benign, 13% DDoS). In contrast, the Friday test split is much closer to balanced, indicating a material distribution shift. This supports the evaluation choice to report macro-averaged metrics and to treat Friday as a deployment-oriented generalisation test rather than an extension of the training window.

4. Experimental Design and Training Strategy

4.1 Evaluation Objectives

The experimental design assesses supervised models for detecting DDoS activity in flow-level traffic under conditions that approximate operational deployment. The evaluation emphasises: (i) robustness to class imbalance, (ii) generalisation under temporal distribution shift, and (iii) computational feasibility at scale.

4.2 Baseline Model

A majority-class dummy classifier is used as a reference point. It predicts the most frequent class in the training data for all samples and establishes a lower bound on meaningful performance, illustrating why accuracy can be misleading under severe class imbalance (He and Garcia 2009).

4.3 Model Selection Rationale

Two supervised learning models with contrasting inductive biases are selected.

- **SGD Logistic Regression (incremental):** a linear classifier trained via stochastic optimisation, suitable for large-scale and streaming contexts, with support for incremental updates via `partial_fit` (Pedregosa et al. 2011).
- **Random Forest (batch on stratified sample):** an ensemble of decision trees that captures non-linear interactions and typically attains strong in-distribution performance, at higher computational cost (Breiman 2001).

This comparison highlights trade-offs between computational efficiency, interpretability, and representational capacity.

4.4 Training Regime and Data Access Strategy

For the linear model, training is performed incrementally by streaming processed partitions from disk and updating the classifier in fixed-size chunks. In contrast, random forests do not support incremental fitting; therefore, a stratified sampling strategy is used to construct a representative subset of the processed training data for batch learning.

4.5 Hyperparameter Configuration

Hyperparameters were chosen to balance performance, training time, and reproducibility, under the constraint that at least one model must support streamed updates on commodity hardware. Table 1 reports the final configuration used in the implementation.

Table 1: Final hyperparameter configuration used for each model.

Model	**Configuration**
SGD Logistic Regression	`loss=log_loss`, `penalty=l2`, `alpha=1e-4`, `learning_rate=optimal`, `max_iter=1` with incremental `partial_fit` updates; features standardised via online `StandardScaler`.
Random Forest (stratified sample)	`n_estimators=200`, `max_depth=None`, `min_samples_split=2`, `min_samples_leaf=1`, `n_jobs=-1`.
Dummy baseline	`strategy=most_frequent`.

This configuration reflects a deliberate trade-off between margin stability (via L2 regularisation and small α in the linear model) and representational capacity (via unpruned trees in the ensemble), prioritising generalisation under temporal shift over in-window optimisation.

4.6 Validation and Test Protocol

Performance is measured on (i) a validation set drawn from the same temporal window as training data and (ii) a temporally separated Friday test set. This simulates a deployment setting where a model trained on historical data is applied to future traffic.

4.7 Performance Metrics

Accuracy is reported for completeness, but the primary metrics are macro-averaged F1, class-specific recall for the malicious class, confusion matrices, and ROC-AUC. These metrics better reflect detection quality under imbalance and allow analysis of the trade-off between missed detections and false alarms.

4.8 Reproducibility and Artefact Preservation

The pipeline is executed end-to-end in a single workflow. Trained models are serialised, metrics are exported as structured artefacts, and figures (confusion matrices, ROC curves, and feature-importance plots) are generated programmatically to ensure results are verifiable and repeatable.

5. Implementation and System Architecture

The implementation follows a modular pipeline:

1. Raw ingestion of CIC IDS 2017 CSV files from a local directory.
2. Column standardisation and numerical feature selection.
3. Streaming preprocessing and persisted intermediate outputs generated during execution.
4. Model training (incremental for SGD, batch sample for RF).
5. Evaluation on validation and temporally separated test.
6. Exported artefacts: serialised models (`models/`), figures (`figures/`), and metrics (`metrics_summary.csv`, `metrics_summary_robust.csv`).

This structure is designed for reproducibility and avoids reliance on notebook memory state.

6. Results

6.1 Summary Metrics

Table 2 summarises the key validation and test results produced by the single-run pipeline. Robust SGD results correspond to a retrained model with the top five random-forest-ranked features removed. Values are taken directly from `metrics_summary.csv` and `metrics_summary_robust.csv`.

Table 2: Validation and Friday test performance. "Robust" retrains SGD after removing the top five RF-ranked features.

Model	Split	Accuracy	F1 macro	ROC-AUC
Dummy baseline	Validation	0.8740	0.4664	–
Dummy baseline	Test (Friday)	0.5227	0.3433	–
SGD Logistic Regression	Validation	0.9710	0.9282	0.9871
SGD Logistic Regression	Test (Friday)	0.8447	0.8388	0.9894
SGD Robust (drop-5)	Validation	0.9622	0.9041	0.9807
SGD Robust (drop-5)	Test (Friday)	0.8386	0.8326	0.9778
Random Forest (sample)	Validation	0.9993	0.9984	0.9999
Random Forest (sample)	Test (Friday)	0.8459	0.8396	0.9779

6.2 Confusion Matrices

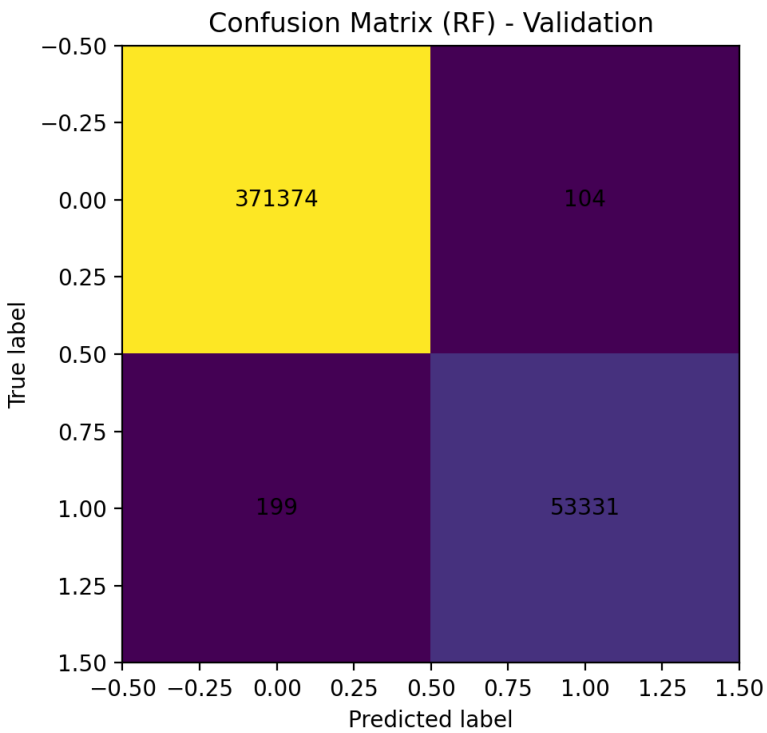

Figure 2: Random forest confusion matrix (validation).

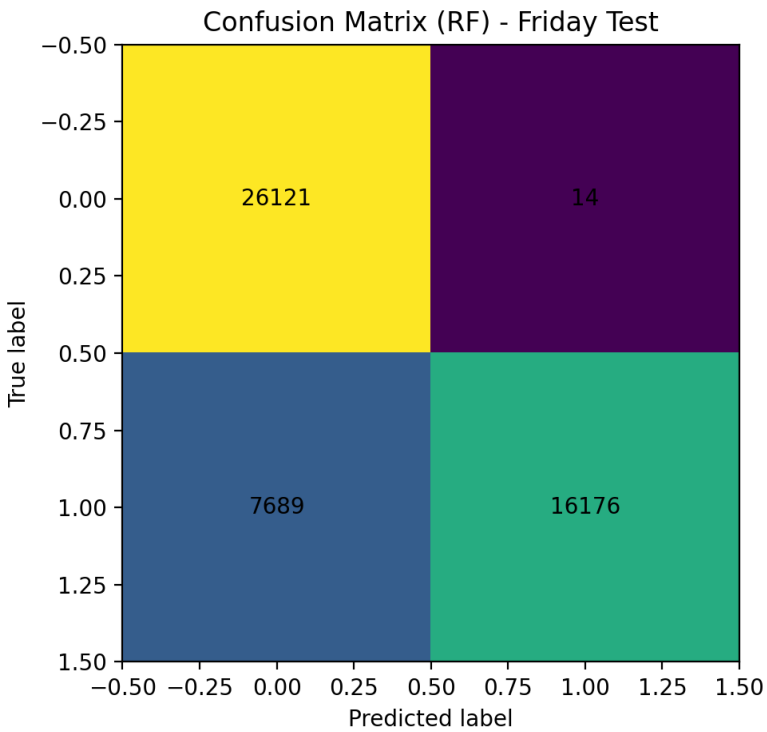

Figure 3: Random forest confusion matrix (Friday test).

6.3 ROC Curve

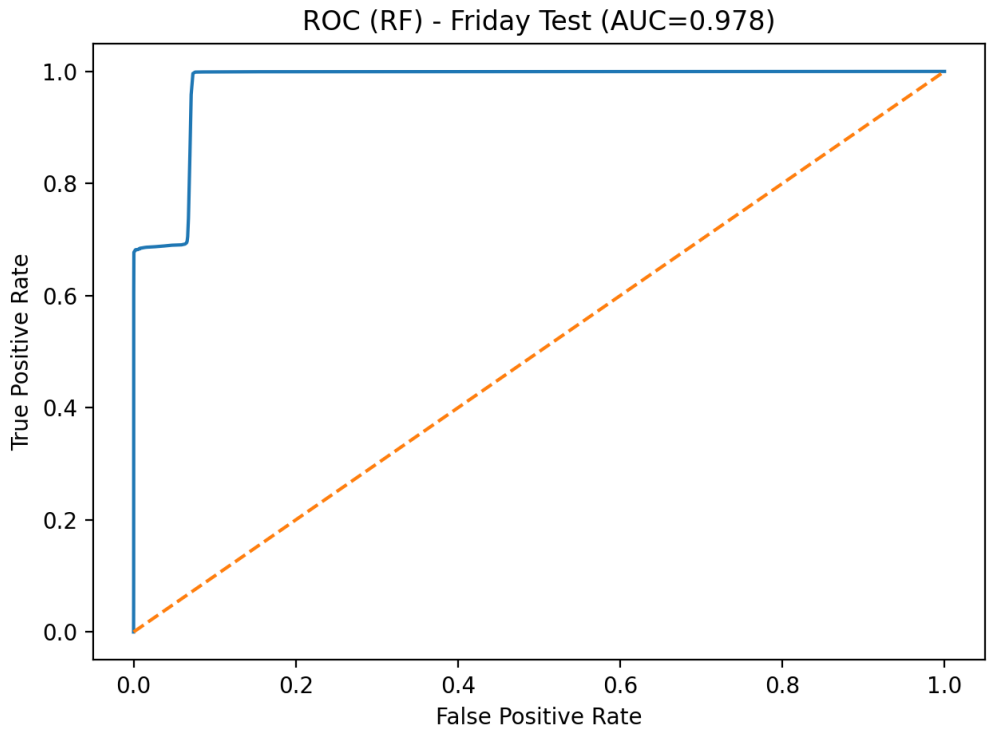

Figure 4: Random forest ROC curve (Friday test).

6.4 Feature Importance and Interpretation

Feature importance for the random forest is shown in Figure 5. Several top-ranked features relate to packet length variation, inter-arrival times, and port behaviour, consistent with expected behavioural changes during volumetric flooding.

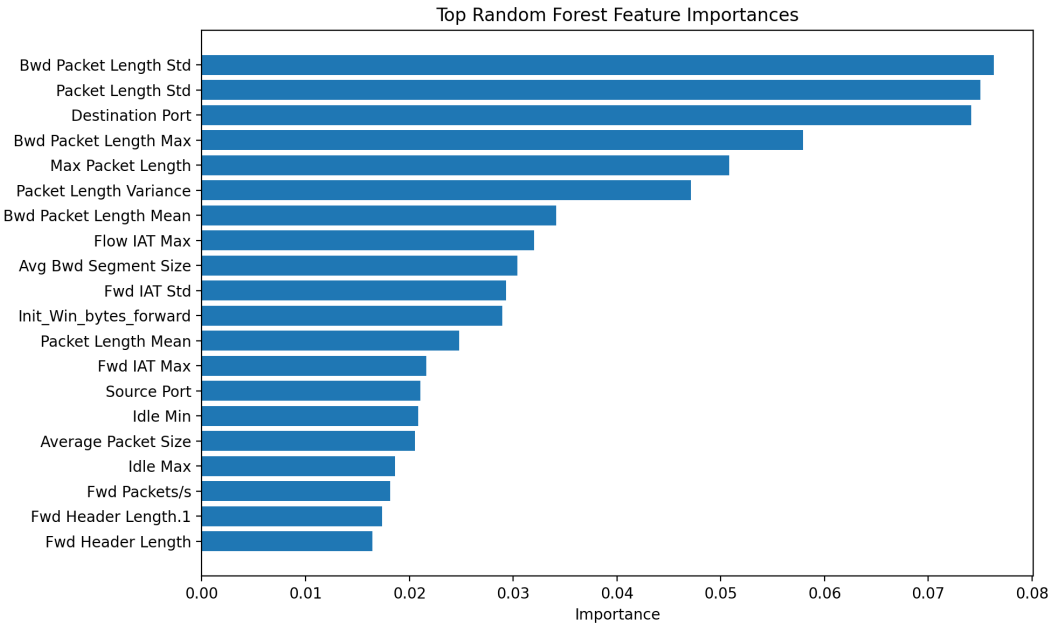

Figure 5: Top feature importances for the random forest model

Feature Robustness Experiment

This experiment tests whether SGD performance depends on a small set of high-influence features. The five highest-ranked random-forest features (*Bwd Packet Length Std*, *Packet Length Std*, *Destination Port*, *Bwd Packet Length Max*, *Max Packet Length*) were removed from the feature set, and the SGD model was retrained using the same streamed training regime.

After removing these features, validation performance decreased relative to the main SGD model (macro F1 from 0.928 to 0.904; ROC-AUC from 0.987 to 0.981), indicating that these variables contribute meaningfully to in-window discrimination. Friday test performance also reduced slightly (macro F1 from 0.839 to 0.833), but remained strong, suggesting the classifier's temporal generalisation is not dependent on any single small feature subset and is supported by a wider set of behavioural signals (Sommer and Paxson 2010).

7. Discussion and Model Comparison

This evaluation tests temporal generalisation: models are trained on Monday–Thursday traffic and assessed on a Friday hold-out as a proxy for deployment drift. The dummy baseline shows why accuracy is unreliable under imbalance, motivating macro-averaged metrics.

The SGD logistic regression model provides a computationally efficient, streaming-compatible classifier with stable generalisation. Its performance indicates that much of the separation between benign and DDoS flows is captured by approximately linear relationships in timing and volume features, supporting its suitability for periodic retraining under resource constraints.

The random forest achieved near-perfect validation performance, consistent with a high-capacity model exploiting complex feature interactions. However, the drop on the Friday test set indicates sensitivity to distribution shift, reflecting a bias–variance trade-off where regularities learned in the training window do not fully persist under changed conditions.

Notably, the random forest model's macro F1 decreases from 0.998 on the validation split to 0.841 on the Friday test set, quantifying the effect of temporal distribution shift.

Overall, the results reinforce a key principle for security ML: the model that looks best on validation data is not necessarily the most reliable under realistic change. The Friday evaluation provides stronger evidence of deployment-relevant performance than a random split of a single capture window (Sommer and Paxson 2010).

8. Feature Analysis and Interpretability

Interpretability matters in security settings, where model outputs often support analyst triage rather than fully automated response. This section interprets feature influence for both modelling approaches, using SGD coefficients and random forest feature importances to infer which behavioural properties are driving classification.

8.1 Linear Model Coefficients (SGD Logistic Regression)

The linear model exposes signed feature coefficients, enabling inspection of which attributes increase or decrease the probability of a flow being classified as malicious. Among the most influential features were inter-arrival time (IAT) statistics and idle or active time measures, which capture temporal dynamics such as burstiness and regularity. These signals align with expectations for volumetric flooding, where traffic exhibits distinctive timing structure compared to user-driven applications.

8.2 Random Forest Feature Importance

Random forests provide impurity-based feature importance scores, indicating which variables most reduce classification uncertainty across the ensemble. The most important features are dominated by packet length statistics and timing measures, including packet length variance, packet length standard deviation, and IAT-related attributes. These features capture higher-order interactions between packet volume and timing, enabling a non-linear decision boundary that is highly effective within the training window.

The importance ranking is useful operationally even when generalisation is imperfect. Consistent emphasis on timing and size features suggests that accurate measurement of these properties is central to DDoS discrimination in flow-based telemetry, and that behavioural signals are more informative than raw identifiers for this task.

8.3 Security Interpretation

Taken together, the feature analyses suggest that both models primarily exploit behavioural characteristics of flows rather than explicit identifiers, which is aligned with practical IDS constraints where payload inspection may be limited by encryption or privacy. However, the presence of some structural features among top contributors highlights a potential evasion avenue: adaptive attackers can modify easily controlled fields to mimic benign patterns.

For this reason, feature influence is interpreted as evidence of dataset-driven discrimination rather than proof of adversary-resistant detection. In deployment, this motivates defence in depth, periodic retraining, and monitoring for drift in feature distributions over time.

9. Limitations and Future Work

While the pipeline demonstrates strong performance on CIC IDS 2017, several limitations constrain direct claims about real-world readiness.

9.1 Simulation Bias and Dataset Representativeness

CIC IDS 2017 is generated in a controlled enterprise-like environment with scripted attacks. This enables ground truth labelling but introduces simulation bias, where statistical regularities may reflect the experimental setup rather than intrinsic properties of malicious behaviour. High-capacity models are especially able to exploit subtle correlations that do not generalise.

9.2 Temporal Shift and Concept Drift

Network behaviour is non-stationary. Changes in workload, infrastructure, and attacker strategy can shift the relationship between features and labels over time. The Friday test set provides a limited proxy for this shift, and the observed performance differences between validation and Friday illustrate why evaluation should not rely solely on in-window splits. Longer-horizon testing and periodic retraining strategies would be required to maintain detection effectiveness under drift (Widmer and Kubat 1996).

9.3 Adversarial Manipulation

Intrusion detection is adversarial. Attackers can manipulate certain features at low cost, particularly ports and superficial protocol fields. Behavioural timing and volumetric features are more costly to spoof, but sophisticated adversaries may throttle or shape traffic to mimic benign patterns. Future work should include robustness experiments that simulate evasion strategies and quantify degradation.

9.4 Scalability Constraints

The pipeline required streaming preprocessing and a sampling strategy for random forest training. This reflects practical constraints, but also introduces a trade-off between representativeness and compute cost. For real-time monitoring, linear models offer advantages due to incremental learning and lower inference overhead, whereas ensembles may require stronger infrastructure or careful deployment design.

9.5 Future Work

Future work includes threshold selection using precision-recall analysis, robustness testing under simple evasion strategies, and evaluation across additional capture windows to better characterise drift and stability.

10. Reproducibility and Artefacts

Key artefacts produced by the pipeline:

- Processed partitions under `data/` (streamed outputs, not included in submission due to size).
- Trained models in `models/` (for example `rf_sample.joblib`, `sgd_logreg_stream.joblib`).
- Figures in `figures/` (confusion matrices, ROC curves, and feature-importance plots).
- Metrics exported as `metrics_summary.csv` and `metrics_summary_robust.csv`.

10.1 How to Reproduce the Results

To reproduce this coursework end-to-end:

1. Place the authoritative CIC IDS 2017 CSV files under `data/`.
2. Run the preprocessing stage (in `dsac_final.ipynb`) to generate cleaned Parquet partitions in a local output directory.
3. Run the final end-to-end notebook cell that trains the dummy baseline, streamed SGD model, and stratified-sample random forest.
4. Confirm outputs are written to `models/`, `figures/`.

If models already exist in `models/`, evaluation can be repeated without retraining by loading the saved artefacts and re-running the evaluation cells.

11. Conclusion

This coursework implemented a reproducible binary intrusion detection pipeline on CIC IDS 2017 using flow-level features, with explicit attention to class imbalance, temporal generalisation, and commodity hardware limits. Streaming preprocessing produced deterministic processed partitions, enabling repeatable training and evaluation with exported artefacts.

Across models, a clear trade-off emerges. SGD logistic regression supports scalable training and stable behaviour under temporal shift, while the random forest achieves near-perfect in-window validation but drops on Friday, reinforcing that strong validation scores are not sufficient evidence of robustness. Overall, the pipeline demonstrates that behavioural flow features can be highly discriminative for DDoS detection, while emphasising the need for drift-aware evaluation and cautious deployment claims.

Appendix A: Figure Index

This appendix enumerates all figures generated by the single-run pipeline. Each figure is produced programmatically and stored under **figures/** to ensure traceability from experimental outputs to reported results.

Filename	Pipeline Stage	Description
`fig\_cm\_rf\_val.png`	Evaluation (Validation)	Confusion matrix for the random forest model on the Monday-Thursday validation split, showing class-specific detection and misclassification rates.
`fig\_cm\_rf\_test.png`	Evaluation (Test)	Confusion matrix for the random forest model on the temporally separated Friday test set, used to assess generalisation under distribution shift.
`fig\_roc\_rf\_test.png`	Evaluation (Test)	Receiver Operating Characteristic (ROC) curve for the random forest model on the Friday test set, illustrating the trade-off between true positive and false positive rates across thresholds.
`fig\_feature\_importance\_rf.png`	Interpretation	Impurity-based feature importance ranking for the random forest model, highlighting the most influential packet-level and timing features used for classification.
`fig\_cm\_sgd\_val.png`	Evaluation (Validation)	Confusion matrix for the SGD logistic regression model on the validation split, used to compare linear and non-linear model behaviour under identical data conditions.
`fig\_cm\_sgd\_test.png`	Evaluation (Test)	Confusion matrix for the SGD logistic regression model on the Friday test set, assessing linear model robustness under temporal distribution shift.

Filename	Pipeline Stage	Description
`fig\_roc\_sgd\_test.png`	Evaluation (Test)	ROC curve for the SGD logistic regression model on the Friday test set, enabling threshold-independent comparison with the random forest classifier.
`fig\_cm\_sgd\_robust\_val.png`	Robustness (Validation)	Confusion matrix for the robust SGD model trained after removal of the top five random-forest-ranked features, evaluated on the validation split.
`fig\_cm\_sgd\_robust\_test.png`	Robustness (Test)	Confusion matrix for the robust SGD model on the temporally separated Friday test set, used to assess feature-dependence under distribution shift.
`fig\_roc\_sgd\_robust\_test.png`	Robustness (Test)	ROC curve for the robust SGD model on the Friday test set, illustrating generalisation performance after suppression of dominant features.

Appendix B: Glossary

Term	Definition
Benign Traffic	Network flow instances representing normal, non-malicious behaviour in the CIC-IDS-2017 dataset.
Binary Classification	A supervised learning task in which each instance is assigned to one of two classes, here benign (0) or DDoS (1).
Class Imbalance	A condition where one class is substantially more frequent than the other. In this work, benign flows dominate the Monday-Thursday splits.
Confusion Matrix	A 2×2 table summarising true positives, false positives, true negatives, and false negatives for a binary classifier.
Distribution Shift	A change in the statistical properties of the data between training and evaluation sets. In this project, Friday traffic exhibits a markedly different class balance from earlier days.
DDoS (Distributed Denial of Service)	A type of network attack that overwhelms a target system or service by generating large volumes of traffic from multiple sources.
Feature	A measurable attribute or variable used as input to a machine learning model, such as packet length statistics or flow duration.
Feature Robustness	The extent to which model performance is preserved when influential input features are removed or perturbed.
Macro-Averaged F1 Score	The unweighted mean of the F1 scores computed separately for each class, used to account for class imbalance.
Parquet Partition	A columnar data file format used here to store processed network flow data in manageable chunks for streaming and memory-safe loading.
Random Forest	An ensemble learning method that combines multiple decision trees trained on random subsets of the data and features to improve predictive performance and stability.
ROC Curve (Receiver Operating Characteristic)	A plot showing the trade-off between true positive rate and false positive rate across different classification thresholds.
ROC-AUC	The area under the ROC curve, representing the probability that a classifier ranks a randomly chosen positive instance higher than a randomly chosen negative one.
Scaler (Standardisation)	A preprocessing step that transforms numeric features to have zero mean and unit variance, improving the stability of gradient-based models.
SGD (Stochastic Gradient Descent)	An optimisation method that updates model parameters incrementally using small batches or individual samples, enabling training on large datasets.

Streaming Evaluation	A procedure in which data is processed in sequential chunks rather than loaded into memory at once, allowing evaluation on large datasets under memory constraints.
Temporal Split	A data partitioning strategy where training and testing sets are separated by time, rather than random sampling, to better approximate deployment conditions.
Validation Set	A subset of the training window used to tune models and assess performance prior to evaluation on the held-out test set.

References

Breiman, Leo (2001). "Random forests". In: *Machine Learning* 45.1, pp. 5–32. DOI: 10.1023/A:1010933404324.

He, Haibo and Edwardo A. Garcia (2009). "Learning from imbalanced data". In: *IEEE Transactions on Knowledge and Data Engineering* 21.9, pp. 1263–1284. DOI: 10.1109/TKDE.2008.239.

Pedregosa, Fabian et al. (2011). "Scikit-learn: Machine learning in Python". In: *Journal of Machine Learning Research* 12, pp. 2825–2830.

Sharafaldin, Iman, Arash Habibi Lashkari, and Ali A. Ghorbani (2018). "Toward generating a new intrusion detection dataset and intrusion traffic characterization". In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116. DOI: 10.5220/0006639801080116.

Sommer, Robin and Vern Paxson (2010). "Outside the closed world: On using machine learning for network intrusion detection". In: *Proceedings of the IEEE Symposium on Security and Privacy*, pp. 305–316. DOI: 10.1109/SP.2010.25.

Widmer, Gerhard and Miroslav Kubat (1996). "Learning in the presence of concept drift and hidden contexts". In: *Machine Learning* 23.1, pp. 69–101. DOI: 10.1007/BF00116900.